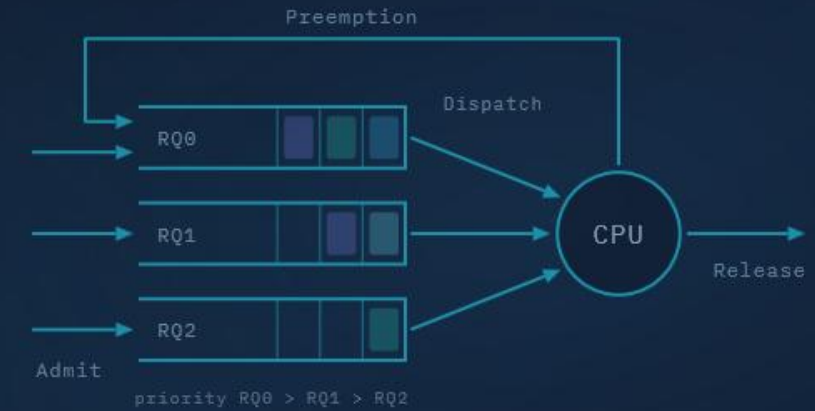


Uniprocessor scheduling

Adiesha Liyana Ralalage



- min turnaround T_r
- T_r / T_s
- min response time
- max throughput
- max CPU utilization
- fairness
- meet deadlines
- $R = (w + s) / s$

FCFS

FIFO - non-preemptive

RR

$q = 1$ - preemptive

SPN

non-preemptive

SRT

preemptive

HRRN

non-preemptive

FB

$q = 1$ - preemptive



Scheduling

- Here we will look at systems with **one processor** and learn about how the scheduling happens for a multiprogramming system.
- The key to **multiprogramming** is **scheduling**.
- The aim of processor scheduling is to assign processes to be executed by the processor or processors over time, in a way that meets system objectives, such as **response time**, **throughput**, and **processor efficiency**.

Types of scheduling

Long-term scheduling	The decision to add to the pool of processes to be executed.
Medium-term scheduling	The decision to add to the number of processes that are partially or fully in main memory.
Short-term scheduling	The decision as to which available process will be executed by the processor.
I/O scheduling	The decision as to which process's pending I/O request shall be handled by an available I/O device.

Types of scheduling

Long-term scheduling	The decision to add to the pool of processes to be executed.
Medium-term scheduling	The decision to add to the number of processes that are partially or fully in main memory.
Short-term scheduling	The decision as to which available process will be executed by the processor.
I/O scheduling	The decision as to which process's pending I/O request shall be handled by an available I/O device.

- **Note: In modern general-purpose desktop operating systems, long, medium, and short scheduling are not necessarily three separate scheduler components.**
- **The reason that we learn about these is that they make fundamentally different decisions.**
 - Long-term: Should this job/process enter the system?
 - Medium-term: Should this process temporarily be removed from active execution?
 - Short-term: Which ready thread should use the CPU now?

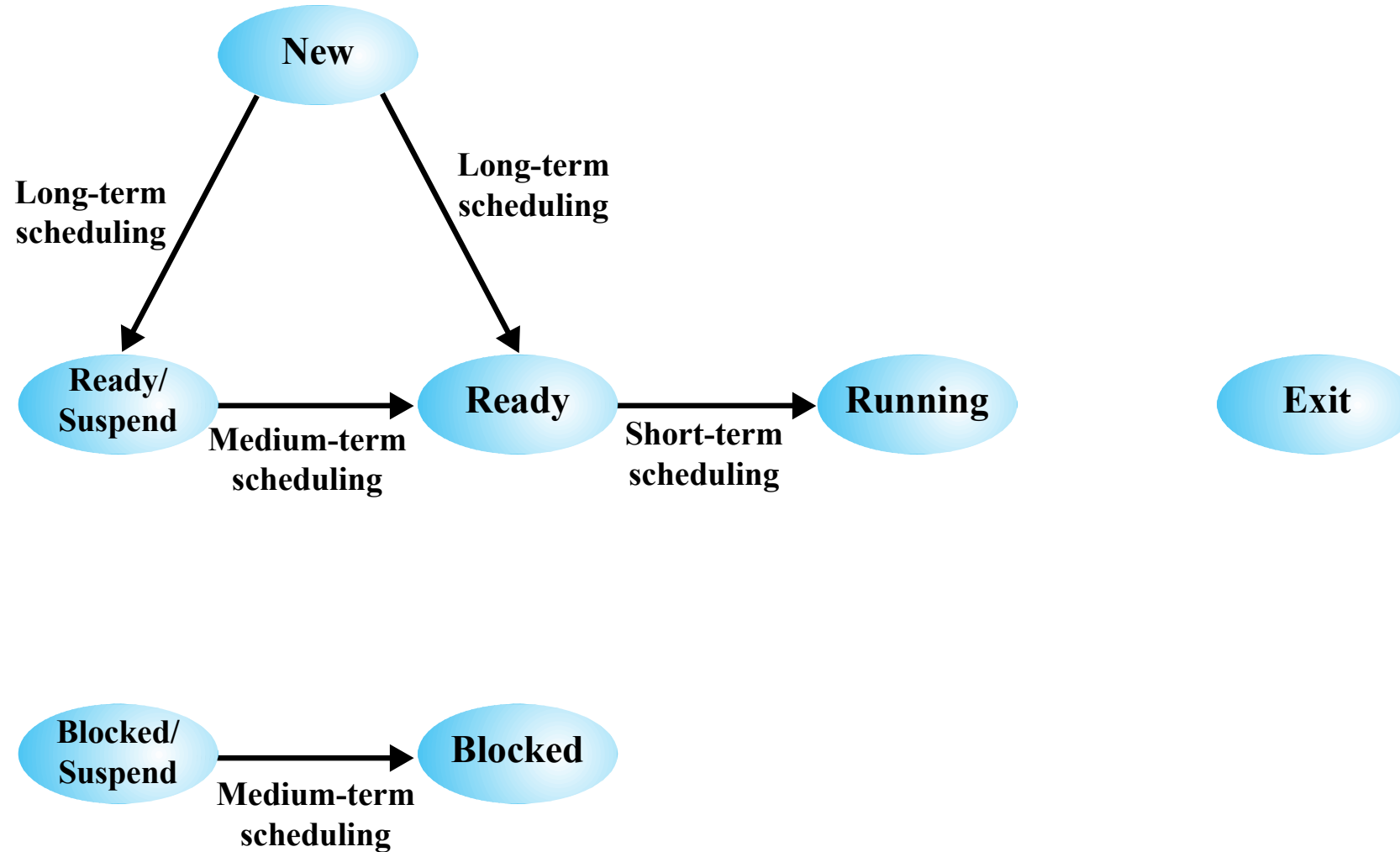


Figure 9.1 Scheduling and Process State Transitions

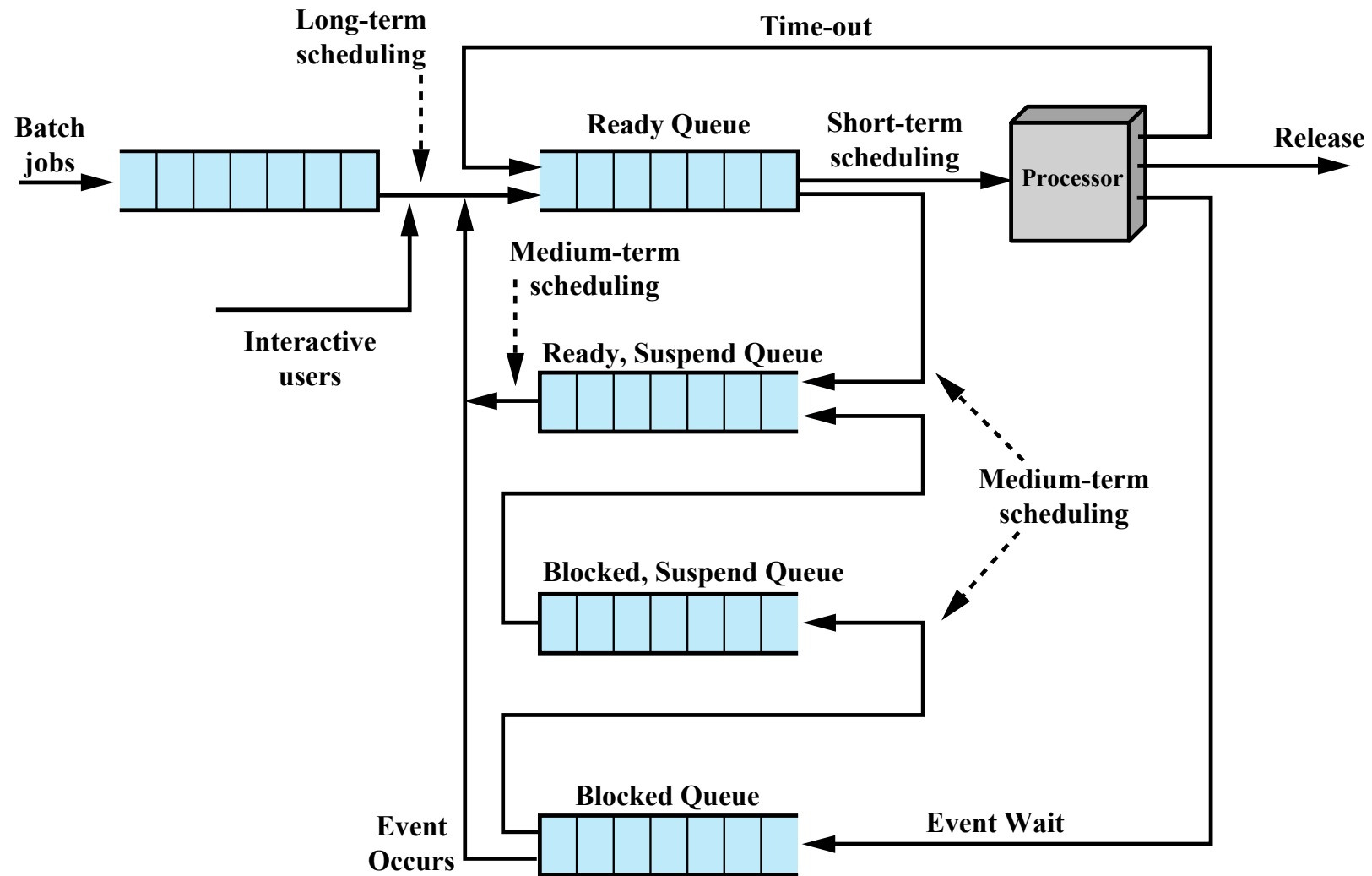
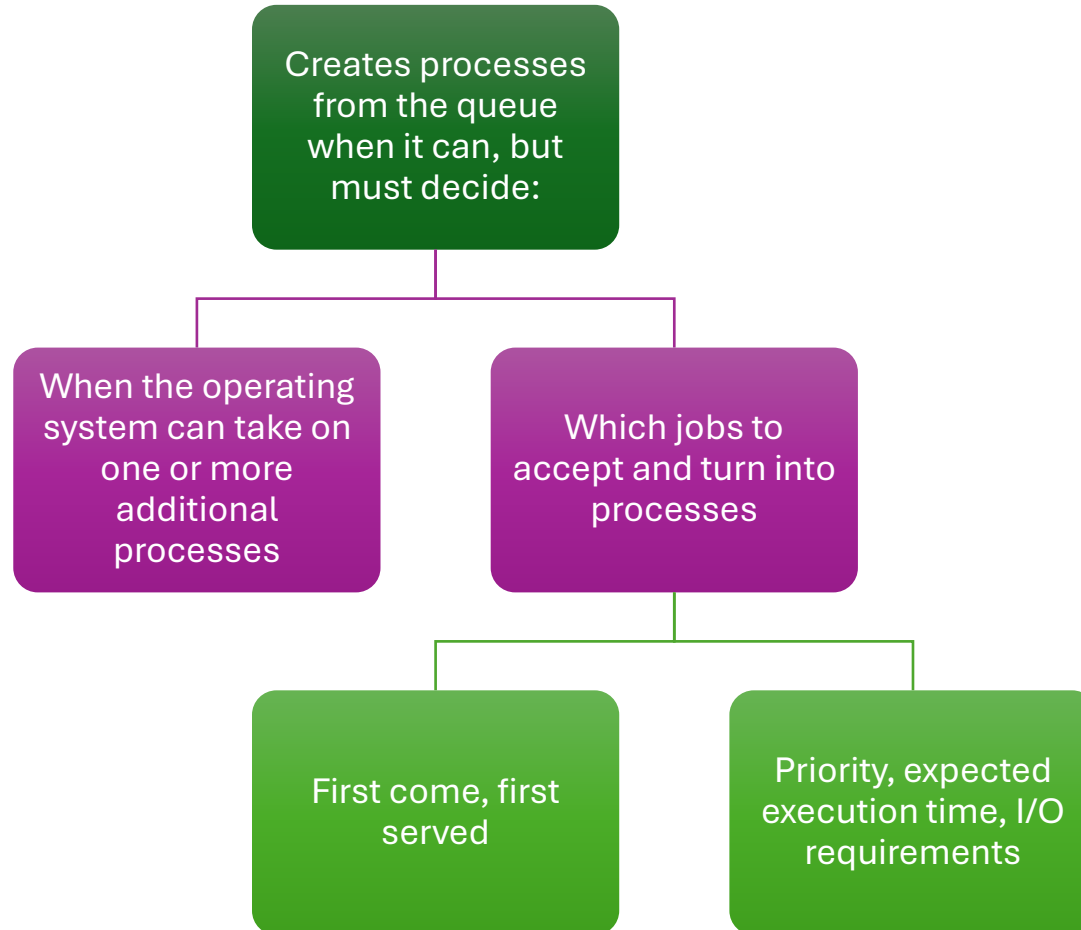


Figure 9.3 Queuing Diagram for Scheduling

Long-term Scheduling

- Long-term scheduling
 - The long-term scheduler determines which programs are admitted to the system for processing.
 - Controls the degree of multiprogramming
 - The more processes that are created, the smaller the percentage of time that each process can be executed
 - May limit to provide satisfactory service to the current set of processes
 - Once admitted, the process is moved to short term scheduler.
 - But in some systems, new programs are created as swapped out processes, in this case, it is moved to the medium-term scheduler.

Long-term Scheduling



Long-Term scheduling

- For interactive programs in a time-sharing system, a process creation request can be generated by the act of a user attempting to connect to the system.
- Time sharing users are not simply queued up and kept waiting until the system can accept them.
- OS will accept all authorized users until the system is saturated.

Medium-term scheduling

- Part of the swapping function.
- Swapping-in decisions are based on the need to manage the degree of multiprogramming
 - Considers the memory requirements of the swapped-out processes

Short-term scheduling

- Known as the dispatcher.
- Executes most frequently.
- Makes the fine-grained decision of which process to execute next.
- Invoked when an event occurs that may lead to the blocking of the current process or that may provide an opportunity to preempt a currently running process in favor of another.

Examples:

- Clock interrupts
- I/O interrupts
- Operating system calls
- Signals (e.g., semaphores)

Scheduling algorithms

- Let's look at scheduling algorithms.
- We are going to focus on short-term scheduling.

Short-term scheduling objective?

- What are we trying to achieve using scheduling?
 - Allocate processor time to processes.
 - Allocate processor time in a way that it will optimize one or more aspects of the system behavior.
 - System behavior can be looked at in two different perspectives.
 - User-oriented
 - System-oriented

Short-term scheduling criteria

User-oriented criteria

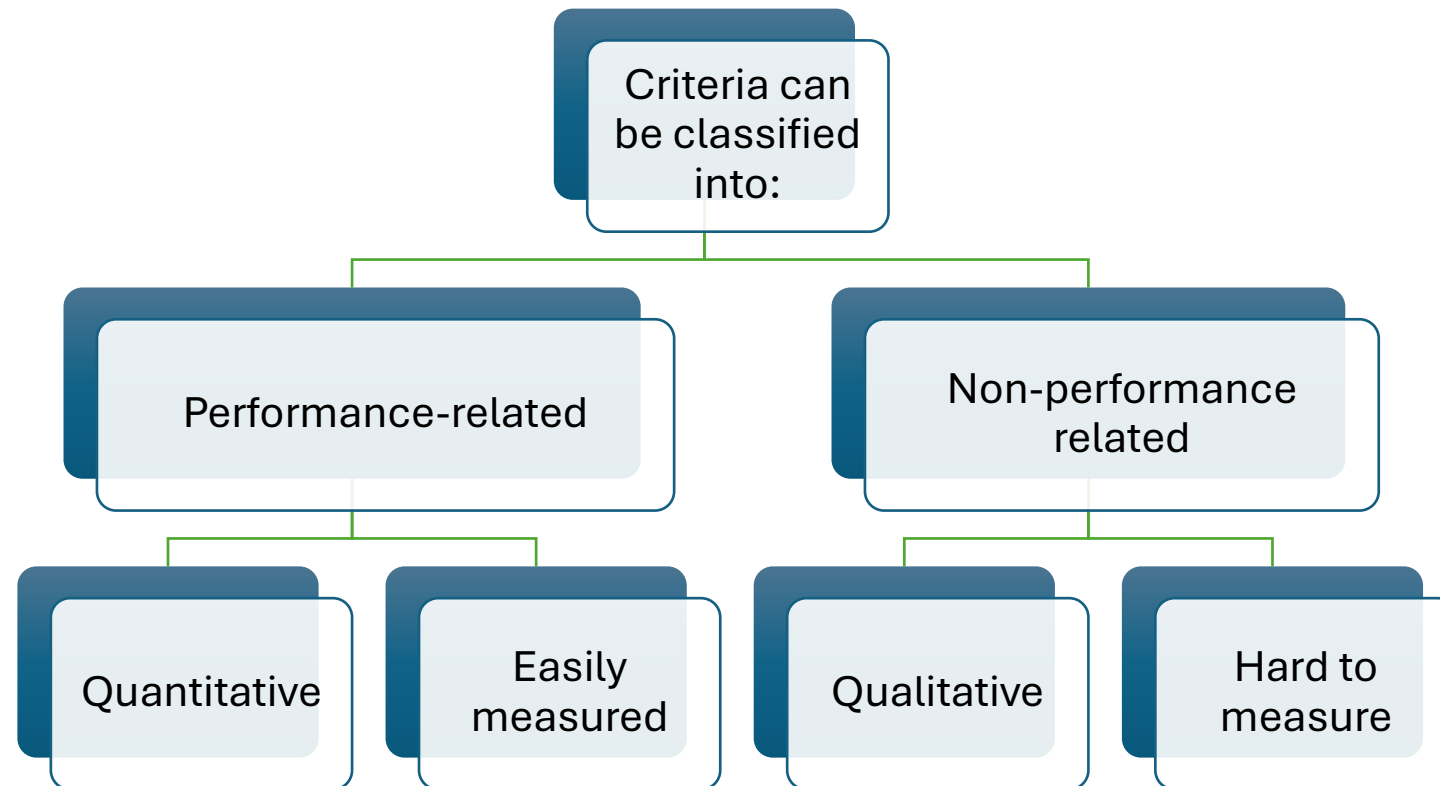
- Relate to the behavior of the system as perceived by the individual user or process (such as response time in an interactive system)
- Important on virtually all systems

System-oriented criteria

- Focus is on effective and efficient utilization of the processor (rate at which processes are completed)
- Generally of minor importance on single-user systems

Short-term scheduling objective?

- Further these criteria can be divided based on performance.



- User oriented and performance related.
 - Turnaround time: interval of time between the submission of a process and its completion.
 - Response time: Time from the submission of a request (for an interactive process) until the response begins to receive.
 - Deadline: when process deadlines can be specified, the scheduling algorithm must fulfil the deadlines or at least the percentage of deadlines.

- User oriented and non-performance related.
 - Predictability: Given a process/job, it should run in the same amount of time and about same cost regardless of the load on the system.

- System oriented and performance related.
 - Throughput: number of processes completed per unit of time.
 - Processor utilization: the percentage of time the processor is busy.

- System oriented and non-performance related.
 - Fairness: Processes should be treated the same in the absence of user or system-supplied guidance.
 - Enforcing priorities: when processes are assigned priorities, the scheduling policy should favor higher-priority processes.
 - Balancing resources: The scheduling policy should keep the system resources busy.

Priority based scheduling

- In many systems, processes are usually assigned a priority.
- Scheduler will pick the process with higher priority to run.
- We can use priority-based queuing approach to implement this.

Priority based scheduling

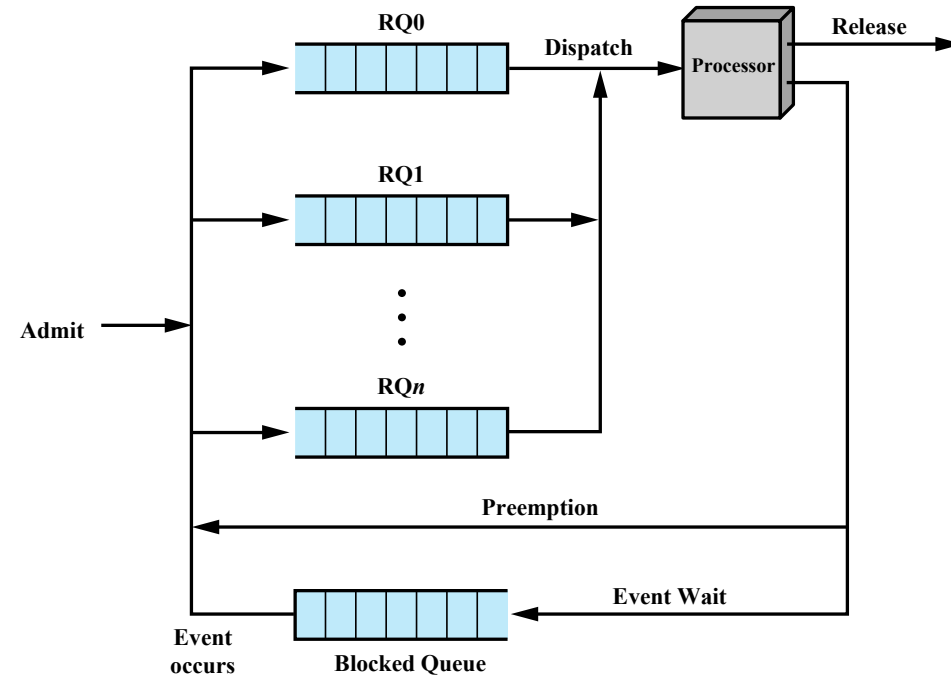


Figure 9.4 Priority Queuing

Priority based scheduling

- Problems?
 - Pure priority-based system may not be a good idea, since there is a chance that lower priority processes may suffer starvation.
 - We can use a scheme where priority of a process changes with its age and execution history.

Scheduling policies

- Now we will look at various scheduling policies.
- Two important things to consider for each policy.
 - **Selection function:** determines which process, among ready processes, is selected next for execution.
 - w = *time spent in the system so far*
 - e = *time spent in the execution so far*
 - s = *total service time required by the process*
 - **Decision mode:** specifies the instants in time at which the selection function is exercised.
 - **Nonpreemptive:** once the process is in running state, it continues to execute until it terminates or blocks itself.
 - **Preemptive:** The currently running process may be interrupted and moved to the ready state by the OS

Nonpreemptive vs preemptive

- Preemptive policies incur greater overhead.
- Provides better services to the total population of processes.
- No monopolizing the CPU by one process
- Efficient process switching, hardware support, larger main memory can help preemptive scheduling policies to run efficiently.
- Most general-purpose modern systems uses preemptive scheduling, however there are operating systems that uses nonpreemptive scheduling policies such as real-time operating systems.

Let's look at an example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

<https://www.cs.montana.edu/adiesha/animations/assets/os/scheduling-comparison.html>

First-Come-First Serve (FCFS)

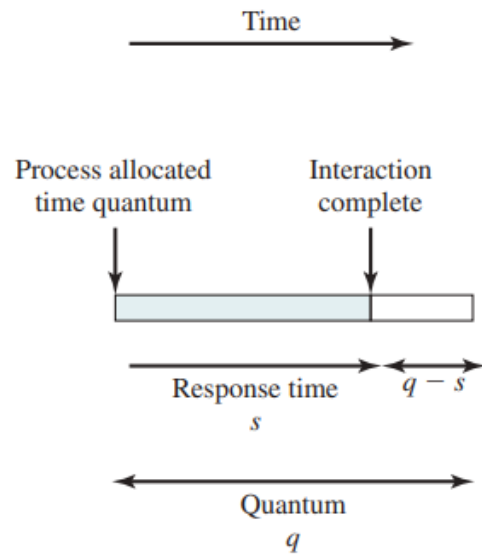
- **Selection function:** $\max[w]$
- **Decision mode:** Non-preemptive
- Simplest scheduling policy.
- As each process becomes ready, it joins the ready queue
- When the currently running process ceases to execute, the process that has been in the ready queue the longest is selected for running.
- How to implement this?
 - Simply use a queue data structure to hold the ready processes

First-Come-First Serve (FCFS)

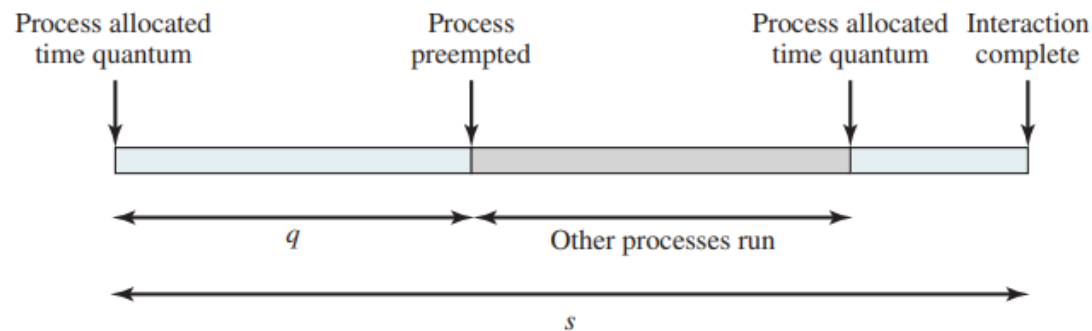
- Performs much better for long processes than short ones.
- Tends to favor processor-bound processes over I/O-bound processes.
 - a long CPU-bound process can occupy the processor for a long time while short I/O-bound processes wait behind it.
 - P1: CPU-bound, needs 100ms of CPU
 - P2: I/O-bound, needs 2ms CPU, then does I/O
 - P3: I/O-bound, needs 2ms CPU, then does I/O

Round Robin

- **Selection function: constant**
- **Decision mode: preemptive (at a given time quantum : q)**
- Uses preemption based on a clock.
- Also known as time slicing because each process is given a slice of time before being preempted.
- A clock interrupt is generated at periodic intervals.
- When the interrupt occurs, the currently running process is placed in the ready queue, and the next ready job is selected on a FCFS basis.
- Let's look at an example.
- Principal design issue is the length of the time quantum, or slice, to be used (q).



(a) Time quantum greater than typical interaction



(b) Time quantum less than typical interaction

- If the quantum is very short, then short processes will move through the system relatively quickly.
- But there is a processing overhead involved with the clock interrupt and the scheduling: Very short time quantum must be avoided.
- One useful guide is that the time quantum should be slightly greater than the time required for a typical interaction or process function.

Round Robin

- Particularly effective in a general-purpose time-sharing system or transaction processing system
- One drawback is its relative treatment of processor-bound and I/O-bound processes.
 - I/O bound processes may not use the full “time quantum”.

Shortest Process Next (SPN)

- **Selection function:** $\min(s)$
- **Decision mode: Non-Preemptive**
- Non-preemptive policy in which the process with the shortest expected processing time is selected next
- A short process will jump to the head of the queue
- Possibility of starvation for longer processes.
- Example $\Rightarrow$
- Variability of response time is increased (less predictable)
- How to implement this?
 - Use a priority queue.

Shortest Process Next (SPN)

- Difficulty: need to know, or at least estimate, the required processing time of each process.
 - OS can use previous statistics stored.
 - User/programmer can indicate this value.
- If the programmer's estimate is substantially under the actual running time, the system may abort the job.
- A risk with SPN is the possibility of starvation for longer processes, as long as there is a steady supply of shorter processes.
- Not suitable for time-sharing systems.

Shortest Process Next (SPN)

- Useful for batch processing systems where user interaction is minimum—often in mainframes and early computers.
- Could be useful in real-time operating systems (RTOS) or embedded systems.

Shortest Remaining Time

- **Selection function:** $\min(s - e)$
- **Decision mode:** Preemptive
- Preemptive version of SPN.
- Scheduler always chooses the process that has the shortest expected remaining processing time (at arrival).
- Example [⇒](#)
- Risk of starvation of longer processes.
- Should give superior turnaround time performance to SPN because a short job is given immediate preference to a running longer job.
- Elapsed service time must be recorded.

Shortest Remaining Time

- Does not favor long processes.
- No additional interrupts are generated (Dispatcher is only invoked when a new process comes to the system—no clock interrupts).
 - Reduced overhead

Highest Response Ratio Next

- **Selection function:** $R = \max \left(\frac{w+s}{s} \right)$ *Ratio = $\frac{\text{time spent waiting} + \text{expected service time}}{\text{expected service time}}$*
- **Decision mode: Non-Preemptive.**
- When the current process completes or is blocked, choose the ready process with the greatest value of R .
- Attractive because it accounts for the age of the process
- While shorter jobs are favored, aging without service increases the ratio so that a longer process will eventually get past competing shorter jobs
- Smaller process are preferred in this approach.

Multilevel Feedback

- **Decision mode: Preemptive**
- **Extension of First come first serve.**
- If we have no indication of the relative length of various processes, then none of SPN, SRT, and HRRN can be used.
- focus on time spent in the execution so far.
- Scheduling done at time quantum.
- Dynamic priority mechanism is used.

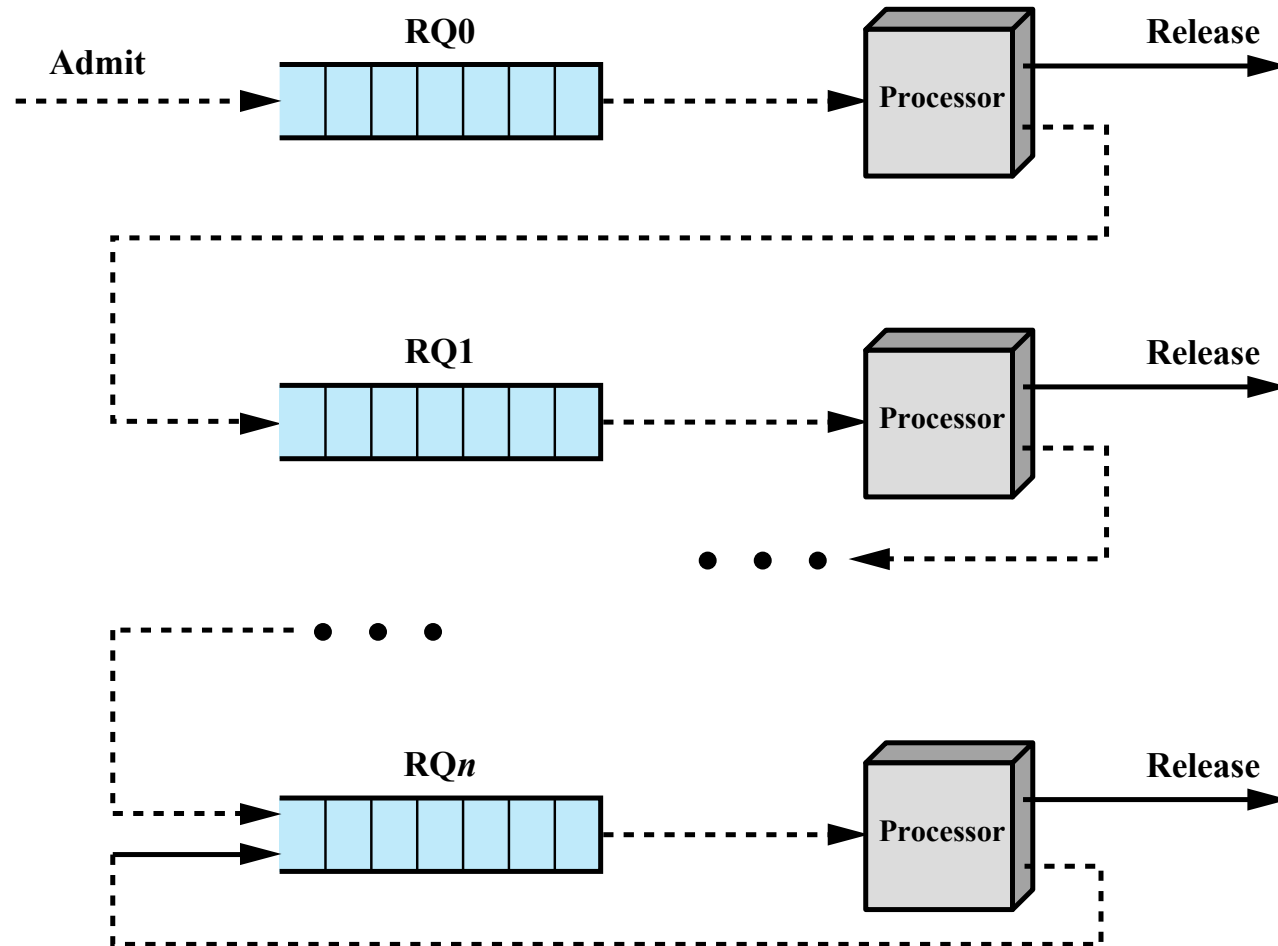
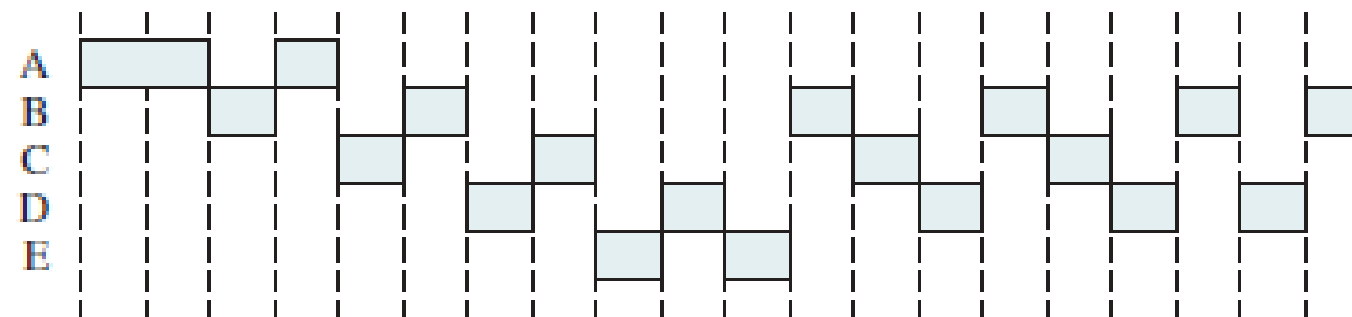


Figure 9.10 Feedback Scheduling

Multilevel Feedback

- When a process first enters the system, it is placed at the Priority 0 queue.
- After it is preempted, it is return to the priority 1 queue.
- Each subsequent time process will be placed to lower priority queue.
- In the lowest priority queue, FCFS mechanism is used.
- You always pick ready processes from highest priority queue, if there is no processes there, you move to the next one and so on.
- However, turnaround time for longer processes stretch out very fast.

Feedback (FB)
 $q = 1$



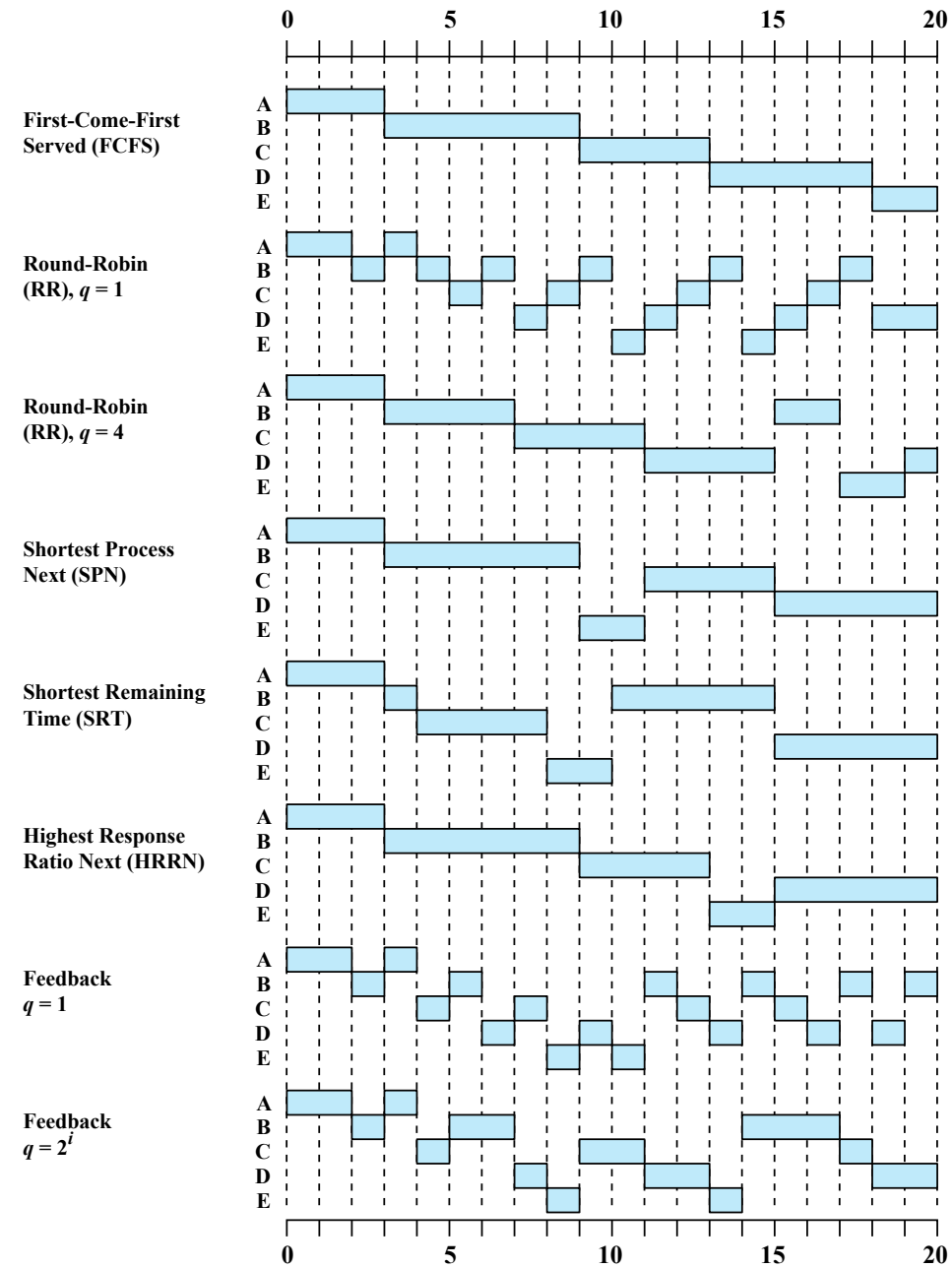


Figure 9.5 A Comparison of Scheduling Policies

Process	A	B	C	D	E	
Arrival Time	0	2	4	6	8	
Service Time (T_s)	3	6	4	5	2	Mean
FCFS						
Finish Time	3	9	13	18	20	
Turnaround Time (T_r)	3	7	9	12	12	8.60
T_r/T_s	1.00	1.17	2.25	2.40	6.00	2.56
RR $q = 1$						
Finish Time	4	18	17	20	15	
Turnaround Time (T_r)	4	16	13	14	7	10.80
T_r/T_s	1.33	2.67	3.25	2.80	3.50	2.71
RR $q = 4$						
Finish Time	3	17	11	20	19	
Turnaround Time (T_r)	3	15	7	14	11	10.00
T_r/T_s	1.00	2.5	1.75	2.80	5.50	2.71
SPN						
Finish Time	3	9	15	20	11	
Turnaround Time (T_r)	3	7	11	14	3	7.60
T_r/T_s	1.00	1.17	2.75	2.80	1.50	1.84
SRT						
Finish Time	3	15	8	20	10	
Turnaround Time (T_r)	3	13	4	14	2	7.20
T_r/T_s	1.00	2.17	1.00	2.80	1.00	1.59
HRRN						
Finish Time	3	9	13	20	15	
Turnaround Time (T_r)	3	7	9	14	7	8.00
T_r/T_s	1.00	1.17	2.25	2.80	3.5	2.14
FB $q = 1$						
Finish Time	4	20	16	19	11	
Turnaround Time (T_r)	4	18	12	13	3	10.00
T_r/T_s	1.33	3.00	3.00	2.60	1.5	2.29
FB $q = 2i$						
Finish Time	4	17	18	20	14	
Turnaround Time (T_r)	4	15	14	14	6	10.60
T_r/T_s	1.33	2.50	3.50	2.80	3.00	2.63